

**Design and Analysis of Algorithms
(CS2009)**

Course Instructor(s):

Ms. Noor ul Ain, Mr. Muhammad Owais Idrees,

Ms. Hira Mastoor

Section(s): (A,B,C,D,E,F,G)-CS

Sessional-I Exam

Total Time (Hrs): 1

Total Marks: 65

Total Questions: 6

Date: Sep 23, 2025

Slution

Roll No

Do not write below this line.

A - C

Course Section

[Signature]

Student Signature

Attempt all the questions.

1. Read the question carefully, understand the question, and then attempt your answers. You can write any assumptions clearly along with the answer if required.
2. **Solve Q1, 2, 3, 4 on the question paper. Solve Q5, 6 on the answer sheet.**
3. Verify that you have **Seven (07)** printed page of the question paper including this page.
4. Use answer sheet for the rough work for questions 1,2,3,4
5. Overwriting and cutting in the final answer on the question paper will result in zero score.
6. Clearly **mark your question number and part number in the answer sheet**. Attempt the questions on your answer sheet in **sequential order**. Answer all parts of questions together.
7. There are two bonus marks if you solve questions in sequential order.
8. Avoid long stories and irrelevant code while answering your question

[CLO 2. Analyze the time and space complexity of different algorithms by using standard asymptotic notations for recursive and nonrecursive algorithms]

Q1: Assume that each of the expressions below gives the processing time $T(n)$ spent by an algorithm for solving a problem of size n . Select the dominant term(s) having the steepest increase in n and specify the Big-Oh time complexity of each algorithm. **[05 marks]**

Expression	Dominant term(s)	$O(\cdot)$
$n^2 \log_2 n + n(\log_2 n)^2$	$n^2 \log_2 n$	$O(n^2 \log_2 n)$
$n \log_3 n + n \log_2 n$	$n \log_2 n$	$O(n \log_2 n)$
$3 \log_8 n + \log_2 \log_2 \log_2 n$	$3 \log_8 n$	$O(\log_8 n)$
$0.01 n \log_2 n + n(\log_2 n)^2$	$n(\log_2 n)^2$	$O(n \log_2^2 n)$
$0.003 \log_4 n + \log_2 \log_2 n$	$0.003 \log_4 n$	$O(\log_4 n)$

[CLO 2. Analyze the time and space complexity of different algorithms by using standard asymptotic notations for recursive and nonrecursive algorithms]

National University of Computer and Emerging Sciences
Islamabad Campus

Q2: You have the following algorithm. Please fill in the following table to find out the functions in terms of the best and worst cases **[10 marks]**

Part A: Fill in the table, enter the number of times the algorithm will run, considering worst-case analysis [2]

Line	Statement	Cost	Times
1	for i = 1 to A.length - 1	c_1	
2	for j = 1 to A.length - i	c_2	
3	if A[j] > A[j+1]	c_3	
4	swap A[j] and A[j+1]	c_4	

Part B: Write the equation of the algorithm derived after the worst-case analysis. [2]

$$O(n^2)$$

Part C: What is the lower bound of the above equation? [2]

$$\Omega(n^2)$$

Part D: The table will change if you consider the best-case scenario. Based on that, rewrite the equation obtained after performing the best-case analysis. [2]

$$O(n)$$

Part E: What is the upper bound of the above equation? [2]

$$O(n^2)$$

National University of Computer and Emerging Sciences
Islamabad Campus

[CLO 2. Analyze the time and space complexity of different algorithms by using standard asymptotic notations for recursive and nonrecursive algorithms]

Q3: You are given some code snippets below. Please provide the time complexity of each snippet in terms of Big-O notation. [3+3+3 = 9 marks]

a. A Smiley Function

```
int smiley(int n) {  
    if (n < 5)  
        return n * n;  
    else {  
        for (int i = 0; i < 10,000; i++) {  
            print i;  
        }  
        return smiley(n / 2);  
    }  
}
```

Time Complexity : $O(\lg n)$

b. A Sunny Function

```
void sunny(int n, int sum) {  
    for (int i = 1; i < n * n; i++) {  
        {  
            for (int j = 0; j < i; j++) {  
                sum++;  
            }  
        }  
    }  
}
```

Time Complexity : $O(n^4)$

c. A Happy Function

```
void happy(int n, int sum) {  
    int k = 1;  
    sum = 1;  
    while (k < n) {  
        for (int i = 0; i < k; i++) {  
            sum++;  
        }  
        k = k * 2;  
    }  
    for (int j = sum * sum; j > 0; j--) {  
        k++;  
    }  
}
```

Time Complexity : $O(n^2)$

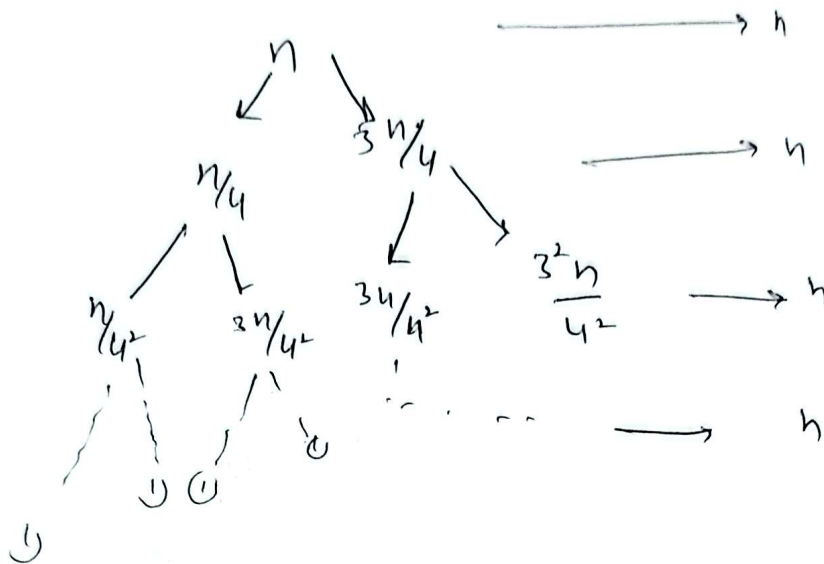
National University of Computer and Emerging Sciences
Islamabad Campus

[CLO 2. Analyze the time and space complexity of different algorithms by using standard asymptotic notations for recursive and nonrecursive algorithms]

Q4: Find the recurrence relation for the given code snippet and determine its time complexity using the Master Theorem. [7+7+7 = 21marks]

Code snippet	Recurrence Relation	Time Complexity
<pre> Fun(n) { if (n<=1) return n sum=0; for (i=2; i<=n; i=i++) for (j=1; j<=i; j=j*2) sum=sum + i*j; for(int i=1; i<=5; i++) sum = sum + (7 * Fun(n/10)) return sum; } </pre>	$T(n) = \begin{cases} 1 & n < 1 \\ 5T(n/10) + n \lg n & n \geq 1 \end{cases}$	$n \lg(n)$
<pre> fun(n) { if (n<=1) return n sum=0; for (i=1; i<=n; i=i*2) sum++ sum=sum+(4+fun(n/4))+(7* fun(n/4)) return sum; } </pre>	$T(n) = \begin{cases} 1 & n < 1 \\ 2T(n/4) + \lg n & n \geq 1 \end{cases}$	Master theorem can't be applied but if (17) one mark deducted
<pre> Fun(n) { if (n<=1) return n sum =2 while(n>1) sum+=n n=n/6 sum = fun(n/6)+fun(n/6)+fun(n/6) return sum; } </pre>	$T(n) = \begin{cases} 1 & n < 1 \\ T(1) + \lg(n) & n \geq 1 \end{cases}$	$\lg(n)$ OR Master Theorem can't work on this

Question No (C):



$$\frac{(h + h + h + \dots + h)}{\log(n) \text{ times}} = \Theta(n \log n)$$

Question No (D):

$$T(n) = T(n-1) + \log(n) \quad \text{--- (*)}$$

$$T(n-1) = T(n-2) + \log(n-1)$$

putting in *

$$T(n) = T(n-2) + \log(n-1) + \log(n) \rightarrow \text{(**)}$$

$$T(n-2) = T(n-3) + \log(n-2)$$

adjusting in (**)

$$T(n) = T(n-3) + \log(n-2) + \log(n-1) + \log(n)$$

$$\begin{aligned} T(n) &= T(n-(n-1)) + \log(n-(n-2)) + \log(n-(n-3)) + \dots + \log(n) \\ &= T(1) + \log(2) + \log(3) + \dots + \log(n-1) + \log(n) \\ &= 1 + \log(1 \times 2 \times 3 \times 4 \times \dots \times n) \\ &= \log(n!) \Rightarrow \log(n)^n = O(n \log n) \end{aligned}$$